



PROJETO E ANÁLISE DE ALGORITMOS

AULA 4 – DIVISÃO E CONQUISTA

4.1 ABORDAGEM DE DIVISÃO E CONQUISTA

- Muitos algoritmos úteis são **recursivos** em sua estrutura: para resolver dado problema, eles chamam a si mesmos recursivamente uma ou mais vezes para lidar com subproblemas intimamente relacionados.
- Em geral, esses algoritmos seguem uma abordagem de divisão e conquista: eles desmembram o problema em vários **subproblemas** que são semelhantes ao problema original, mas de menor tamanho, resolvem os subproblemas recursivamente e depois **combinam** essas soluções com o objetivo de encontrar uma solução para o problema original.

4.1 ABORDAGEM DE DIVISÃO E CONQUISTA

- No método de divisão e conquista, se o problema for pequeno o suficiente — o **caso-base** —, ele é resolvido diretamente, sem necessidade de recursão. Caso contrário — o **caso recursivo** —, são realizadas três etapas características:
 - **Divisão** do problema em um ou mais subproblemas, que são instâncias menores do problema original.
 - **Conquista** dos subproblemas, resolvendo-os recursivamente.
 - **Combinação** das soluções dadas aos subproblemas na solução para o problema original.

4.1 ABORDAGEM DE DIVISÃO E CONQUISTA

- O algoritmo de ordenação por intercalação obedece rigorosamente o método de divisão e conquista.
- Em cada etapa, ele ordena um subvetor $A[p : r]$, começando com o vetor inteiro $A[1 : n]$ e realizando a recursão para subvetores **cada vez menores**.

4.1 ABORDAGEM DE DIVISÃO E CONQUISTA

- **Divisão** do subvetor $A[p : r]$ para ser ordenado em dois subvetores adjacentes, cada um com metade do tamanho total. Para fazer isso, calcule o ponto médio q de $A[p : r]$ (tomando a média de p e r) e divida $A[p : r]$ em subvetores $A[p : q]$ e $A[q + 1 : r]$.
- **Conquista** ordenando os dois subvetores $A[p : q]$ e $A[q + 1 : r]$ recursivamente, utilizando a ordenação por intercalação.
- **Combinação** intercalando os dois subvetores ordenados $A[p : q]$ e $A[q + 1 : r]$ e gerando $A[p : r]$ de volta, para produzir a resposta ordenada.

4.1 ABORDAGEM DE DIVISÃO E CONQUISTA

- A recursão “se esgotará” — alcançará o caso-base — quando o subvetor $A[p : r]$ a ser ordenado tiver **apenas um elemento**, ou seja, quando p for igual a r .
- Como foi observado no argumento de inicialização para o invariante de laço do ORDENA-POR-INSERÇÃO, um subvetor compreendendo apenas um único elemento sempre está ordenado.
- A operação-chave do algoritmo de ordenação por intercalação ocorre na etapa “combinar”, que intercala dois **subvetores adjacentes e ordenados**.

4.1 ABORDAGEM DE DIVISÃO E CONQUISTA

- A operação de intercalação é realizada pelo procedimento auxiliar $\text{MERGE}(A, p, q, r)$, em que A é um vetor e p, q e r são índices de enumeração dos elementos do vetor, tais que $p \leq q < r$.
- O procedimento considera que os subvetores adjacentes $A[p : q]$ e $A[q + 1 : r]$ já foram ordenados recursivamente.
- Ele intercala (ou mescla) os dois subvetores ordenados para formar um **único subvetor ordenado**, que substitui o subvetor atual $A[p : r]$.

4.1 ABORDAGEM DE DIVISÃO E CONQUISTA



4.1 ABORDAGEM DE DIVISÃO E CONQUISTA

$nL = q - p + 1$ // tamanho de $A[p : q]$

$nR = r - q$ // tamanho de $A[q + 1 : r]$

sejam $L[0 : nL - 1]$ e $R[0 : nR - 1]$ novos vetores

4.1 ABORDAGEM DE DIVISÃO E CONQUISTA

```
for i = 0 to nL - 1 // copia A[p:q] para L[0:nL-1]  
    L[i] = A[p + i]
```

```
for j = 0 to nR - 1 // copia A[q+1:r] para R[0:nR-1]  
    R[j] = A[q + j + 1]
```

4.1 ABORDAGEM DE DIVISÃO E CONQUISTA

$i = 0$ // i indexa o menor elemento restante em L

$j = 0$ // j indexa o menor elemento restante em R

$k = p$ // k indexa o local em A para preencher

4.1 ABORDAGEM DE DIVISÃO E CONQUISTA

```
while i < nL e j < nR
    if L[i] ≤ R[j]
        A[k] = L[i]
        i = i + 1
    else
        A[k] = R[j]
        j = j + 1
    k = k + 1
```

//Enquanto cada um de L e R contém um elemento não intercalado, copia o menor elemento não intercalado de volta a A[p : r].

4.1 ABORDAGEM DE DIVISÃO E CONQUISTA

```
while i < nL
```

```
    A[k] = L[i]
```

```
    i = i + 1
```

```
    k = k + 1
```

```
while j < nR
```

```
    A[k] = R[j]
```

```
    j = j + 1
```

```
    k = k + 1
```

// Tendo percorrido
L ou R totalmente,
copia o restante do
outro para o final
de A[p : r].

4.1 ABORDAGEM DE DIVISÃO E CONQUISTA

	8	9	10	11	12	13	14	15	16	17
A	...	2	4	6	7	1	2	3	5	...
	k									

	0	1	2	3
L	2	4	6	7
	i			

	0	1	2	3
R	1	2	3	5
	j			

(a)

	8	9	10	11	12	13	14	15	16	17
A	...	1	4	6	7	1	2	3	5	...
	k									

	0	1	2	3
L	2	4	6	7
	i			

	0	1	2	3
R	1	2	3	5
	j			

(b)

	8	9	10	11	12	13	14	15	16	17
A	...	1	2	6	7	1	2	3	5	...
	k									

	0	1	2	3
L	2	4	6	7
	i			

	0	1	2	3
R	1	2	3	5
	j			

(c)

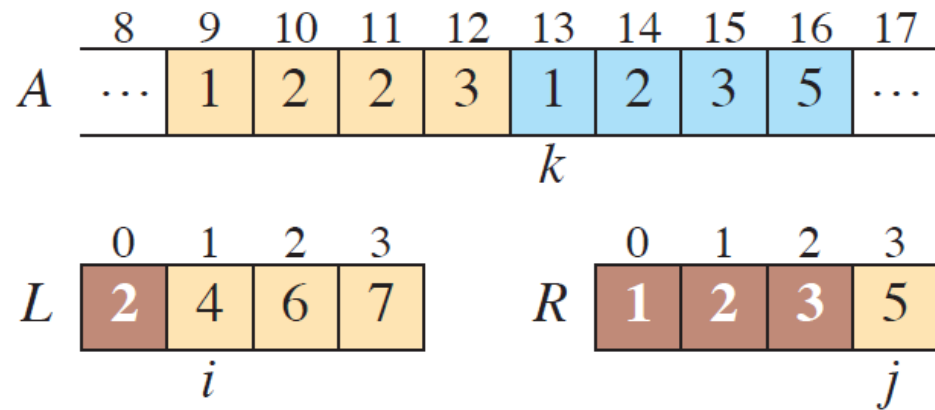
	8	9	10	11	12	13	14	15	16	17
A	...	1	2	2	7	1	2	3	5	...
	k									

	0	1	2	3
L	2	4	6	7
	i			

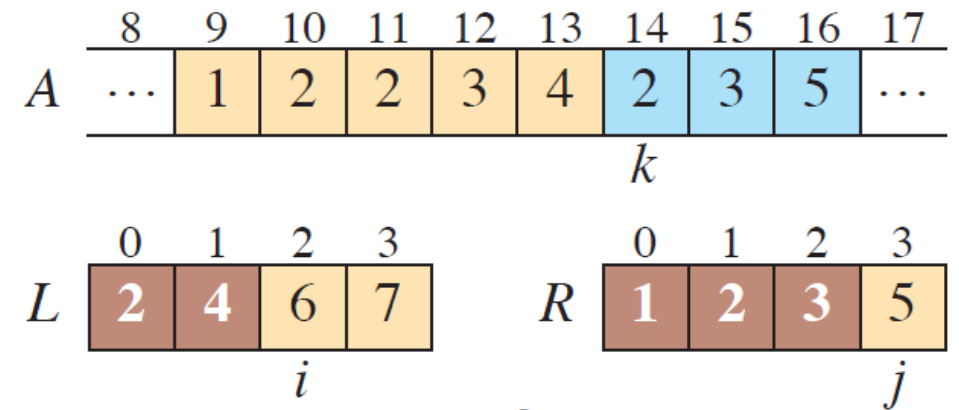
	0	1	2	3
R	1	2	3	5
	j			

(d)

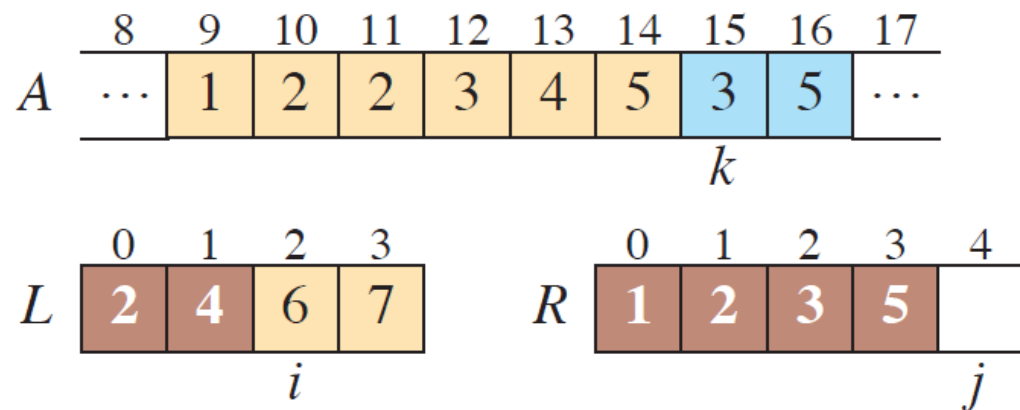
4.1 ABORDAGEM DE DIVISÃO E CONQUISTA



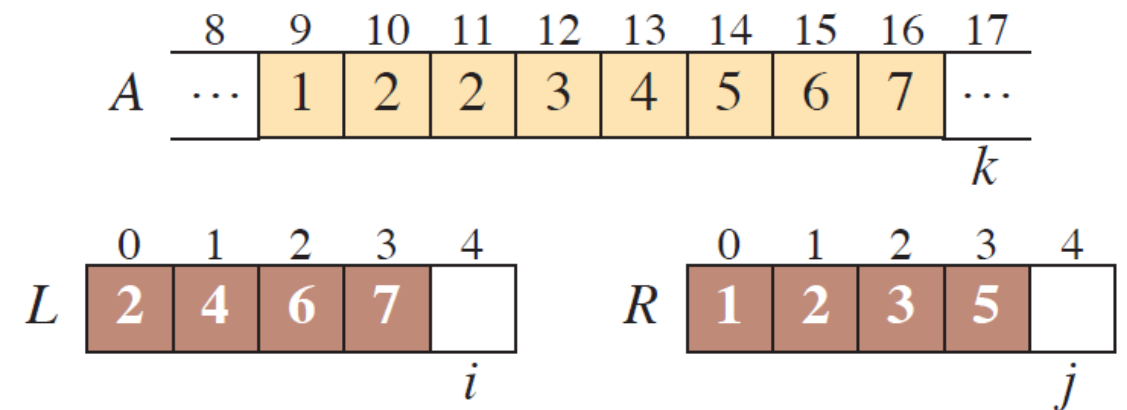
(e)



(f)



(g)



(h)

4.1 ABORDAGEM DE DIVISÃO E CONQUISTA

- O procedimento MERGE é executado no tempo $\Theta(n)$, em que $n = r - p + 1$. As linhas de atribuição e criação dos vetores demora um tempo constante e os laços for que copiam os elementos do vetor original para os subvetores L e R demoram o tempo $\Theta(nL + nR) = \Theta(n)$.
- No caso dos três laços while, cada iteração desses laços copia exatamente um valor de L ou de R para A, e que cada valor é copiado de volta para A exatamente uma vez. Portanto, esses três laços juntos resultam em um **total de n iterações**. Como cada iteração de cada um desses três laços demora um tempo constante, o tempo total gasto nesses três laços é $\Theta(n)$.

4.1 ABORDAGEM DE DIVISÃO E CONQUISTA

- O procedimento MERGE é usado como uma sub-rotina no algoritmo de ordenação por intercalação.
- O procedimento MERGE-SORT(A, p, r) ordena os elementos do subvetor $A[p : r]$. Se p é igual a r , o subvetor tem apenas 1 elemento e, portanto, já está ordenado.
- Caso contrário, devemos ter $p < r$, e o MERGE-SORT executa as etapas de divisão, conquista e combinação.

4.1 ABORDAGEM DE DIVISÃO E CONQUISTA

- A etapa de divisão simplesmente calcula um índice q que subdivide $A[p : r]$ em dois subvetores adjacentes: $A[p : q]$, contendo $\lfloor n/2 \rfloor$ elementos, e $A[q + 1 : r]$, contendo $\lfloor n/2 \rfloor$ elementos.
- A chamada inicial $\text{MERGE-SORT}(A, 1, n)$ ordena o vetor $A[1 : n]$ inteiro.

4.1 ABORDAGEM DE DIVISÃO E CONQUISTA

```
if  $p \geq r$  // zero ou um elemento?
```

```
    return
```

```
q =  $\lfloor (p + r) / 2 \rfloor$  // ponto médio de A[p : r]
```

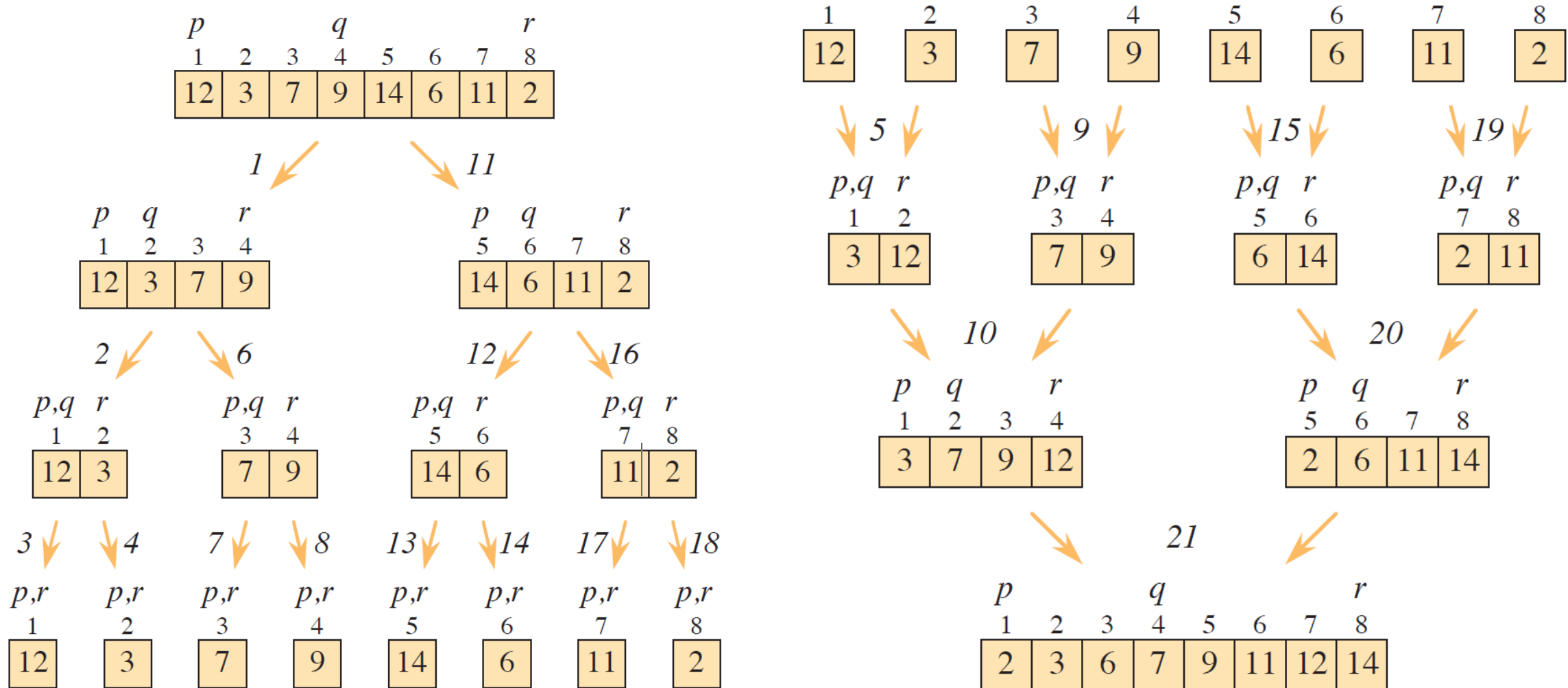
```
MERGE-SORT(A, p, q) // ordena A[p : q] recursivamente
```

```
MERGE-SORT(A, q + 1, r) //ordena A[q+1:r] recursivamente
```

```
// Intercala A[p : q] e A[q + 1 : r] em A[p : r].
```

```
MERGE(A, p, q, r)
```

4.1 ABORDAGEM DE DIVISÃO E CONQUISTA



4.2 ANÁLISE DE ALGORITMOS DE DIVISÃO E CONQUISTA

- Quando um algoritmo contém uma chamada recursiva, seu tempo de execução pode ser descrito frequentemente por uma equação de recorrência ou recorrência, que descreve o tempo de execução global para um problema de tamanho n em termos do tempo de execução para entradas menores. Então, podemos usar ferramentas matemáticas para resolvermos a recorrência e estabelecermos limites para o desempenho do algoritmo.
- Uma recorrência para o tempo de execução de um algoritmo de divisão e conquista resulta dos três passos do método básico.

4.2 ANÁLISE DE ALGORITMOS DE DIVISÃO E CONQUISTA

- Consideramos $T(n)$ o tempo de execução do pior caso para um problema de tamanho n .
- Se o tamanho do problema for pequeno o bastante, digamos $n \leq n_0$ para alguma constante $n_0 > 0$, a solução direta demorará um tempo constante, que representamos por $\Theta(1)$.
- Vamos supor que a subdivisão que adotamos para o problema produza **a** subproblemas, cada um deles com **n/b** , ou seja **$1/b$** do tamanho do problema original.

4.2 ANÁLISE DE ALGORITMOS DE DIVISÃO E CONQUISTA

- No caso da ordenação por intercalação, **a e b são 2**.
- O algoritmo leva o tempo $T(n/b)$ para resolver um subproblema de tamanho n/b e, portanto, leva o tempo **$aT(n/b)$** para resolver um número **a** desses problemas.
- Se o algoritmo levar o tempo $D(n)$ para dividir o problema em subproblemas e o tempo $C(n)$ para combinar as soluções dadas aos subproblemas na solução para o problema original, obteremos a recorrência a seguir.

4.2 ANÁLISE DE ALGORITMOS DE DIVISÃO E CONQUISTA

$$T(n) = \begin{cases} \theta(1) & \text{se } n < n_0 \\ D(n) + aT\left(\frac{n}{b}\right) + C(n) & \text{caso contrário} \end{cases}$$

- Às vezes, o tamanho n/b da etapa de divisão não é um número inteiro. Por exemplo, o procedimento MERGE-SORT divide um problema de tamanho n em subproblemas de tamanhos $\lceil n/2 \rceil$ e $\lfloor n/2 \rfloor$. Como a diferença entre $\lceil n/2 \rceil$ e $\lfloor n/2 \rfloor$ é no máximo 1, o que para n grande é muito menor do que o efeito de dividir n por 2, vamos forçar um pouco a barra e apenas chamar os dois de tamanho $n/2$.

4.2 ANÁLISE DE ALGORITMOS DE DIVISÃO E CONQUISTA

- Análise da ordenação por intercalação:
 - **Divisão:** a etapa de divisão simplesmente calcula o ponto médio do subvetor, o que demora um tempo constante. Portanto, $D(n) = \Theta(1)$.
 - **Conquista:** resolvemos recursivamente dois subproblemas, cada um de tamanho $n/2$, o que contribui com $2T(n/2)$ para o tempo de execução (ignorando os pisos e tetos).
 - **Combinação:** o procedimento MERGE em um subvetor de n elementos leva o tempo $\Theta(n)$; assim, $C(n) = \Theta(n)$.

4.2 ANÁLISE DE ALGORITMOS DE DIVISÃO E CONQUISTA

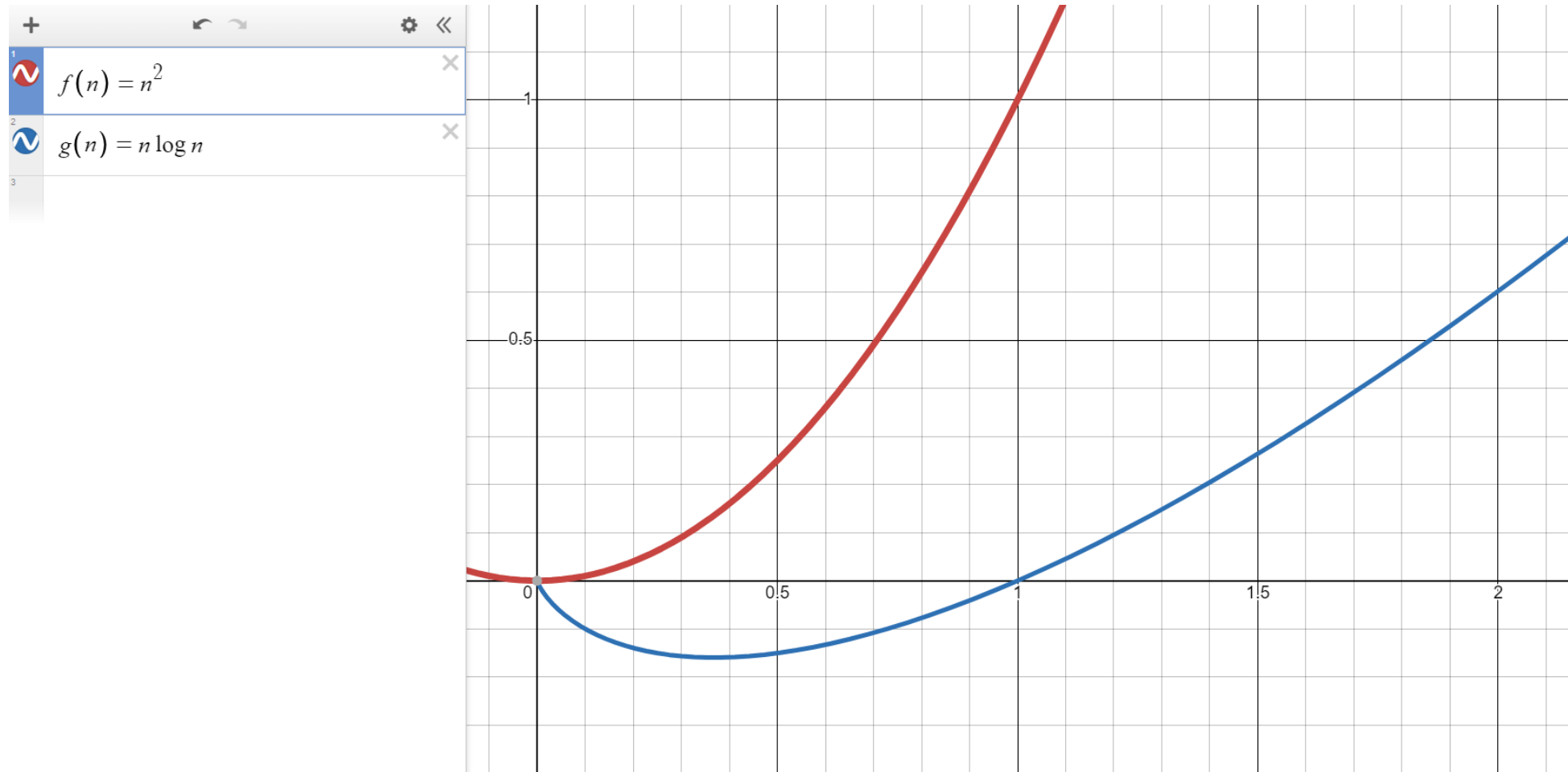
- Quando somamos as funções $D(n)$ e $C(n)$ para a análise da ordenação por intercalação, estamos somando uma função que é $\Theta(n)$ e uma função que é $\Theta(1)$.
- Essa soma é uma **função linear** de n . Ou seja, ela é aproximadamente proporcional a n quando n é grande, e portanto, os tempos de divisão e combinação da ordenação por intercalação são $\Theta(n)$. A adição de $\Theta(n)$ ao termo $2T(n/2)$ da etapa de conquista fornece a recorrência para o tempo de execução do pior caso $T(n)$ da ordenação por intercalação como a seguir.

4.2 ANÁLISE DE ALGORITMOS DE DIVISÃO E CONQUISTA

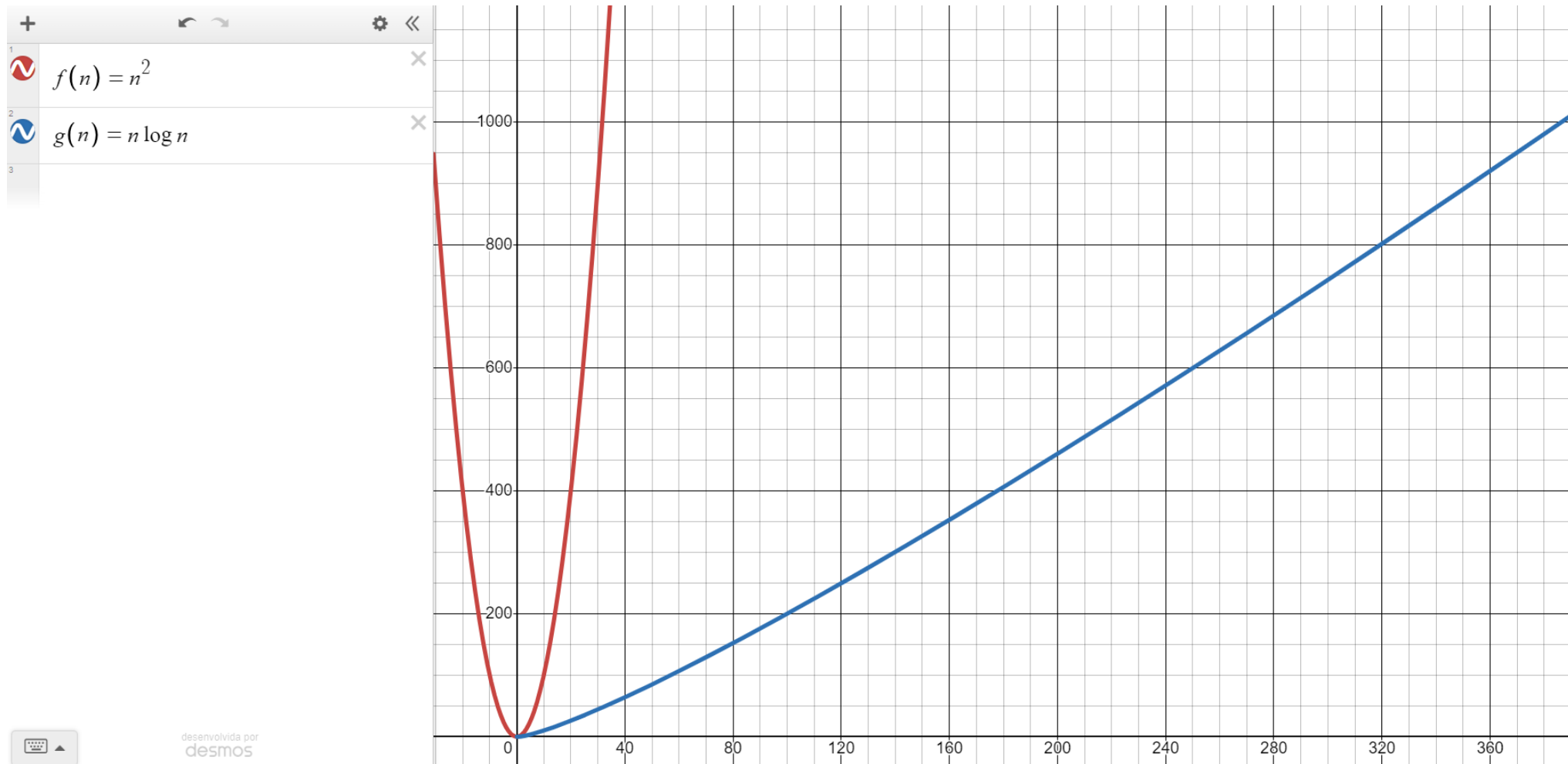
$$T(n) = 2T\left(\frac{n}{2}\right) + \theta(n)$$

- Essa recorrência tem como solução $\Theta(n \lg n)$.
- Em comparação com a ordenação por inserção, cujo tempo de execução do pior caso é $\Theta(n^2)$, a ordenação por intercalação troca um fator de n por um fator de $\lg n$. Como a função logarítmica cresce mais lentamente do que qualquer função linear, essa troca é um bom negócio. Para entradas suficientemente grandes, a ordenação por intercalação, com seu tempo de execução $\Theta(n \lg n)$ do pior caso, supera a ordenação por inserção, cujo tempo de execução é $\Theta(n^2)$, no pior caso.

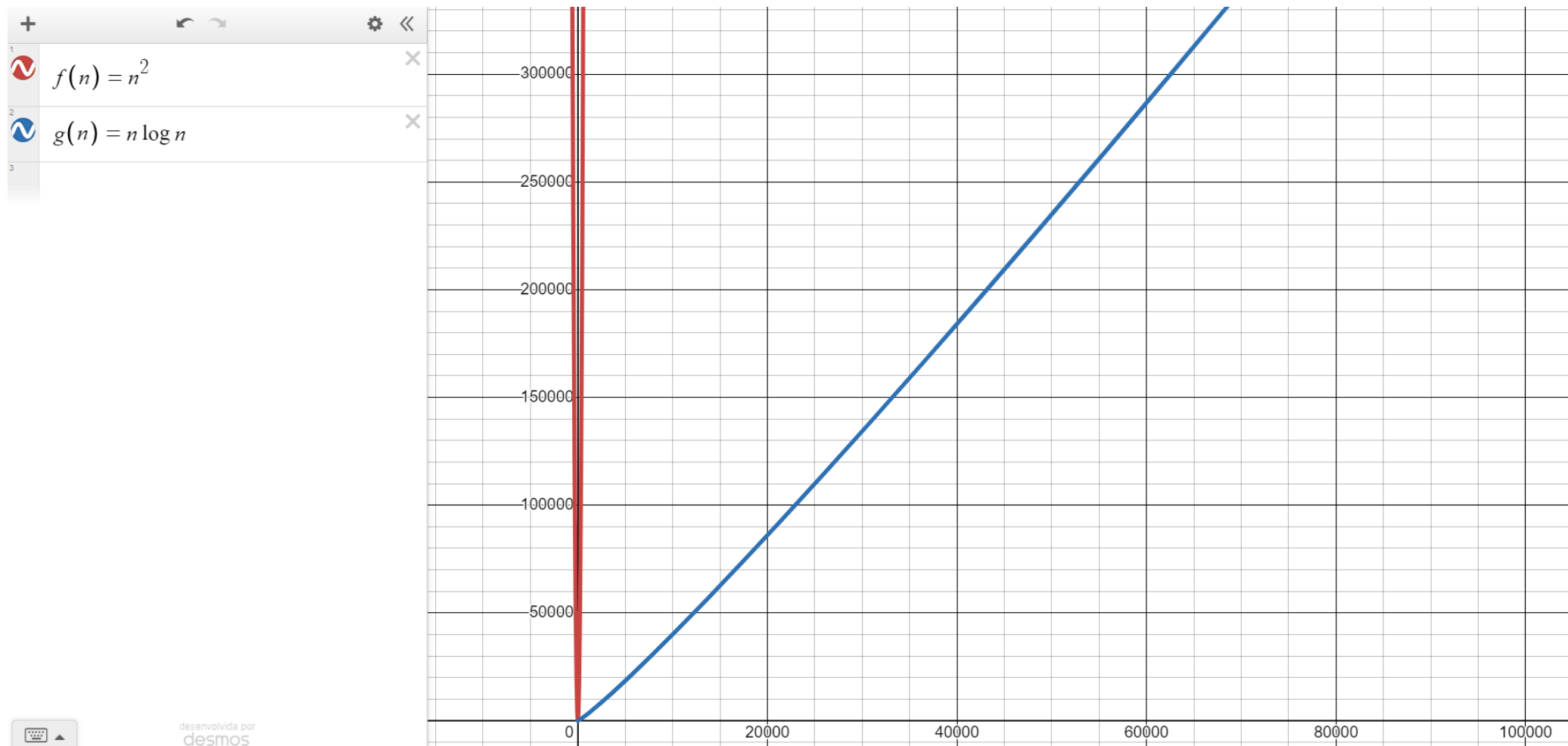
4.2 ANÁLISE DE ALGORITMOS DE DIVISÃO E CONQUISTA



4.2 ANÁLISE DE ALGORITMOS DE DIVISÃO E CONQUISTA



4.2 ANÁLISE DE ALGORITMOS DE DIVISÃO E CONQUISTA



4.2 ANÁLISE DE ALGORITMOS DE DIVISÃO E CONQUISTA

$$T(n) = 2T\left(\frac{n}{2}\right) + \theta(n)$$

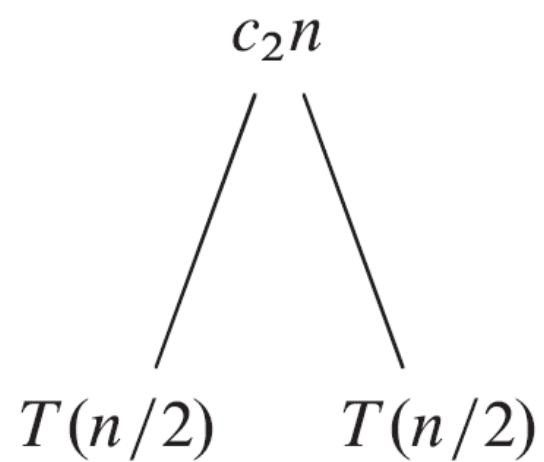
- Essa recorrência tem como solução $\Theta(n \lg n)$.
- O raciocínio para esse resultado começa com supor que n seja uma potência exata de 2 e que o caso-base implícito seja $n = 1$. Assim:

$$T(n) = \begin{cases} c_1 & \text{se } n = 1, \\ 2T\left(\frac{n}{2}\right) + c_2n & \text{se } n > 1 \end{cases}$$

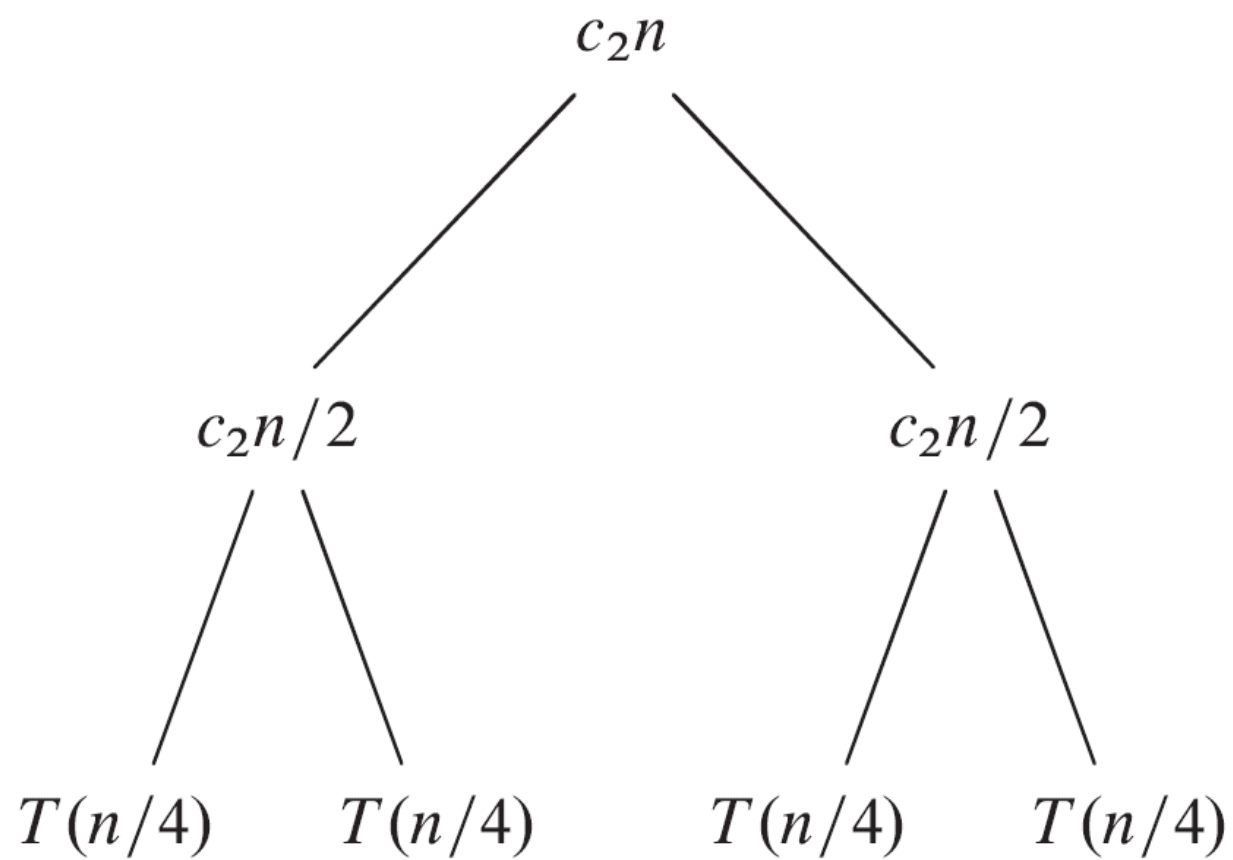
4.2 ANÁLISE DE ALGORITMOS DE DIVISÃO E CONQUISTA

- em que a constante $c_1 > 0$ representa o tempo exigido para resolver problemas de tamanho 1, e $c_2 > 0$ é o tempo por elemento do vetor para as etapas de divisão e combinação.
- Podemos usar uma árvore de recursão para entender como a recorrência se “desdobra”. Simulamos nessa árvore as expansões que ocorrem, isto é, as chamadas recursivas ao longo da execução do algoritmo.
- Sabemos que, para um algoritmo correto, eventualmente as chamadas recursivas terminam, e isso é representado pelas folhas da árvore.

$T(n)$

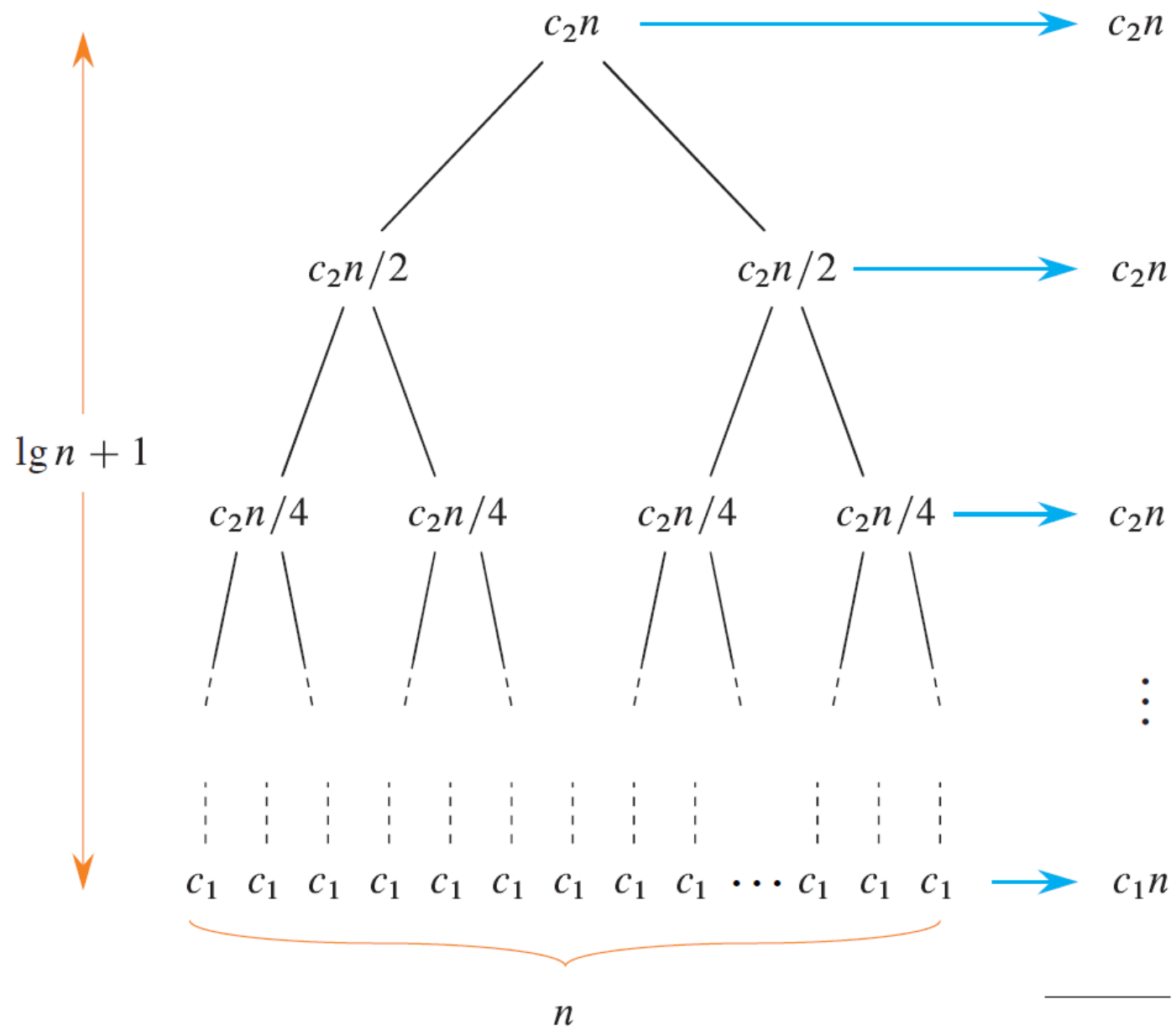


(a)



(b)

(c)



Total: $c_2n \lg n + c_1n$

4.2 ANÁLISE DE ALGORITMOS DE DIVISÃO E CONQUISTA

- Considerando os custos de cada nível da árvore somados
 - O nível superior tem custo total c_2n , o próximo nível abaixo tem custo total $c_2(n/2) + c_2(n/2) = c_2n$, o nível após esse tem custo total $c_2(n/4) + c_2(n/4) + c_2(n/4) + c_2(n/4) = c_2n$, e assim por diante.
 - Cada nível tem o dobro do número de nós de cada nível acima, mas cada nó contribui somente com metade do custo de um nó do nível superior. De um nível para o próximo, dobrar e dividir ao meio cancela um ao outro, de modo que o custo por cada nível é o mesmo: c_2n .

4.2 ANÁLISE DE ALGORITMOS DE DIVISÃO E CONQUISTA

- Em geral, o nível i abaixo do topo tem 2^i nós, cada qual contribuindo com um custo $c_2(n/2^i)$, de modo que o i -ésimo nível abaixo do topo tem custo total $2^i \cdot c_2(n/2^i) = c_2n$.
- O nível inferior tem n nós, cada um contribuindo com um custo c_1 , para um custo total c_1n .
- O número total de níveis da árvore de recursão é $\lg n + 1$, em que n é o número de folhas, correspondente ao tamanho da entrada.

4.2 ANÁLISE DE ALGORITMOS DE DIVISÃO E CONQUISTA

- O caso- base ocorre quando $n = 1$ e, nesse caso, a árvore tem só um nível.
- Como $\lg 1 = 0$, temos que $\lg n + 1$ dá o número correto de níveis.
- Agora suponha, como hipótese indutiva, que o número de níveis de uma árvore de recursão com 2^i folhas seja $\lg 2^i + 1 = i + 1$ (visto que, para qualquer valor de i , temos que $\lg 2^i = i$).
- Como estamos supondo que o tamanho da entrada é uma potência de 2, o tamanho da próxima entrada a considerar é 2^{i+1} .

4.2 ANÁLISE DE ALGORITMOS DE DIVISÃO E CONQUISTA

- Uma árvore com 2^{i+1} folhas tem um nível a mais que uma árvore de 2^i folhas, e então o número total de níveis é $(i + 1) + 1 = \lg 2^{i+1} + 1$.
- Para calcularmos o custo total representado pela recorrência, simplesmente somamos os custos de todos os níveis.
- A árvore de recursão tem $\lg n + 1$ níveis. Os níveis acima das folhas têm um custo $c_2 n$ cada, e o nível de folha tem um custo $c_1 n$, que nos dá o custo total $c_2 n \lg n + c_1 n = \Theta(n \lg n)$.

4.2 ANÁLISE DE ALGORITMOS DE DIVISÃO E CONQUISTA

- Exercício: use a indução matemática para mostrar que, quando $n \geq 2$ é uma potência exata de 2, a solução da recorrência:

$$T(n) = \begin{cases} 2 & \text{se } n = 2, \\ 2T\left(\frac{n}{2}\right) + n & \text{se } n > 2 \end{cases}$$

4.2 ANÁLISE DE ALGORITMOS DE DIVISÃO E CONQUISTA

- O caso base é quando $n = 2$, e temos $n \lg n = 2 \lg 2 = 2 \times 1 = 2$.
- Para a etapa indutiva, nossa hipótese indutiva é que $T(n/2) = (n/2) \lg(n/2)$. Então:

$$\begin{aligned} T(n) &= 2T(n/2) + n \\ &= 2(n/2)\lg(n/2) + n \\ &= n(\lg n - 1) + n \\ &= n\lg n - n + n \\ &= n\lg n \end{aligned}$$

4.2 ANÁLISE DE ALGORITMOS DE DIVISÃO E CONQUISTA

- Exercício: descreva um algoritmo que, dado um conjunto S de n inteiros e um outro inteiro x , determine se existem ou não dois elementos em S cuja soma seja exatamente x . Seu algoritmo deverá levar um tempo $\Theta(n \lg n)$ no pior caso.